



**Природо-математическа гимназия
„Васил Друмев“ град Велико Търново**

ДИПЛОМНА РАБОТА

за придобиване на трета степен на професионална квалификация

**Тема: Изграждане на комплекс от методи, средства и
подходи при разработване на приложение „Конна база“**

Ученик:

Мартин Кременов Петков

Ръководител консултант:

Деян Друмев

Професия: „Приложен програмист“

Специалност: „Приложно програмиране“

Велико Търново, 2025 г.

Съдържание

ЧАСТ 1. УВОД	4
ЧАСТ 2. ТЕХНОЛОГИИ ЗА РЕАЛИЗИРАНЕ НА ПРОЕКТА.....	5
2.1 ОСОБЕНОСТИ НА УЕБ-БАЗИРАНЕТО ИНФОРМАЦИОННА СИСТЕМА.....	5
2.2 ЕЗИЦИ ЗА РЕАЛИЗИРАНЕ НА УЕБ ПРИЛОЖЕНИЯ	6
2.2.1 HTML (HyperText Markup Language).....	7
2.2.2 CSS (Cascading Style Sheets)	9
2.2.3 JavaScript	11
2.2.4 C#.....	12
2.2.5 ASP.NET Core	13
2.3 МОДЕЛИ И ЕЗИЦИ ЗА БАЗА ДАННИ И СУБД	15
2.3.2 SQL (Structured Query Language)	17
2.3.3 СУБД (Система за управление на бази данни).....	19
ЧАСТ 3. РЕАЛИЗАЦИЯ НА ПЛАН ЗА ИЗГОТВЯНЕ НА УЕБ-СТРАНИЦА ЗА КОННА БАЗА	20
3.1 КОНЦЕПЦИЯ	20
3.2 DESIGN PATTERN	20
3.3 MVC DESIGN PATTERN	20
3.4 МОДЕЛИ В ПРИЛОЖЕНИЕТО „КОННА БАЗА“	21
3.4.1 Потребители (AspNetUsers).....	22
3.4.2 Кон	23
3.4.3 Порода	24
3.4.4 Резервации	25
3.4.5 Диаграма на базата данни	26
3.5 КОНТРОЛЕРИ	27
3.5.1 HomeController	28
3.5.2 HorseController	29
3.5.3 UsersController.....	36
3.5.4 ReservationController	42
ЧАСТ 4. ЗАКЛЮЧЕНИЕ	45
ЧАСТ 5. ИЗПОЛЗВАНИ ИЗТОЧНИЦИ.....	46
ЧАСТ 6. РАБОТА С ПРИЛОЖЕНИЕТО „КОННА БАЗА“	47
6.1 НАЧАЛНА СТРАНИЦА	47
6.2 СТРАНИЦА ЗА РЕГИСТРАЦИЯ	48
6.3 СТРАНИЦА ЗА ВЛИЗАНЕ	49

6.4 ЛИСТ С КОНЕ	50
6.5 ДЕТАЙЛИ НА КОНЯ.....	52
6.6 РЕЗЕРВАЦИЯ НА КОН	54
6.7 УПРАВЛЕНИЕ НА РЕЗЕРВАЦИИТЕ.....	55

Част 1. Увод

Уеб-базираните платформи стават все по-разпространени, тъй като осигуряват лесен достъп до информация независимо от местоположението на потребителя. Те не изискват специализирани инсталации и могат да се използват директно през уеб браузър. Въпреки удобството, създаването на такива системи може да бъде сложно, тъй като изисква внимание към сигурността и добрата потребителска практика.

В началния етап проектът ще изследва съществуващите технологии и методи за разработка на уебсайтове за конни бази. Ще се проведе анализ на необходимите функционалности и ще бъдат определени ключовите характеристики на бъдещия сайт. Това включва проучване на процеса на създаване на потребителски акаунти, управление на информацията за конете и предлаганите услуги, резервация на уроци или разходки.

След анализа и дефинирането на изискванията ще бъде разработена архитектурата на уебсайта, като се вземат предвид добрите практики за сигурност, мащабируемост и издръжливост. За дизайна ще бъде приложен иновативен подход, който ще внесе нови идеи в сферата и ще осигури уникално потребителско изживяване. Целта е да се създаде визуално атрактивен и интуитивен интерфейс, който да отговаря на съвременните технологични тенденции и нуждите на потребителите.

След завършването на архитектурата и дизайна ще започне етапът на разработка, в който уебсайтът ще бъде създаден с помощта на съвременни технологии и програмни езици. Процесът ще включва детайлно изграждане, задълбочени тестове и усъвършенствана оптимизация, за да се гарантират максимална стабилност, висока производителност и безпроблемна работа в различни условия.

Този дипломен проект е насочен към разработването на уебсайт, предназначен за конни бази, като акцентът ще бъде поставен върху съвременните технологични изисквания и удобството за потребителите. В рамките на проекта ще бъде проучен и приложен целият процес на създаване на платформата – от концепцията до реализацията. Очакваният краен резултат е интуитивен,

естетически издържан и напълно функционален уебсайт, който ще улесни управлението на базите и предоставянето на услуги на клиентите.

Част 2. ТЕХНОЛОГИИ ЗА РЕАЛИЗИРАНЕ НА ПРОЕКТА

2.1 Особенности на уеб-базираните информационна система

Уеб информационните системи представляват модерни софтуерни платформи, които използват интернет технологии за доставяне на информация и услуги на потребители или други системи. Те се основават на принципи на хипертекст, което позволява динамично публикуване и актуализиране на данни в реално време.

Тези системи обикновено се състоят от различни компоненти, които взаимодействат помежду си чрез уеб приложения. Web браузърите действат като интерфейс за потребителя (front-end), а базата данни обработва и съхранява информацията (back-end), което позволява стабилна и ефективна работа на системата.

С напредъка на интернет технологиите, уебсайтовете вече не се ограничават до статични HTML страници. Те се трансформират в сложни уеб-базирани информационни системи (WIS), които предоставят достъп до обширни бази данни и услуги, включително в областта на електронния бизнес и управлението на взаимоотношенията с клиенти.

Разработката на уеб-базирани информационни системи поставя нови изисквания пред програмистите. Те трябва да се справят с разнообразието от мрежови условия и различните типове данни, като същевременно осигуряват добра функционалност и използваемост на платформите. За разлика от традиционните софтуерни решения, тези системи са независими от операционни системи и устройства, което ги прави много по-гъвкави и достъпни.

Основното предизвикателство при изграждането на тези системи е разбирането и удовлетворяването на различните изисквания на потребителите, които могат да идват от различни региони с разнообразни нужди и предпочитания. Този процес изисква внимателен анализ и гъвкав подход при събирането на изискванията и тяхното интегриране в системата.

2.2 Езици за реализиране на уеб приложения

Скриптовите езици бързо набраха популярност и се утвърдиха като универсални програмни езици благодарение на тяхната гъвкавост и ефективност. Те са широко използвани в ситуации, когато е по-важно времето за разработка да бъде кратко, отколкото времето за изпълнение на програмата. Те намират приложение в различни области, особено в разработката на динамични уеб приложения, където те играят основна роля.

Един от основните им параметри е вграждането в уеб страници. Скриптовете не се компилират предварително, а се интерпретират в реално време от брауъра при зареждането на страницата. Това дава на разработчиците възможността да извършват промени в съдържанието и визуалното оформление на уеб страниците в отговор на действията на потребителя, чрез манипулиране на елементите на документа с помощта на DOM (Document Object Model).

Важно е да се разбере, че скриптовите езици често се наричат "езици за сценарии" заради основната им роля в автоматизирането на задачи, които обикновено биха изисквали ръчно изпълнение от потребителя. Сценарий представлява поредица от действия или операции, които се изпълняват в определен ред, за да се постигне желаната цел. Тези действия обикновено се интерпретират динамично, което означава, че те се изпълняват по време на изпълнението, без предварително компилиране.

Програмите, написани с помощта на скриптови езици, често автоматизират рутинни задачи, като например валидиране на форми или анимиране на елементи на страницата. Тъй като скриптовете се интерпретират на летящо по време на изпълнението, те обикновено работят по-бавно в сравнение с компилираните езици, които се транслират директно в машинен код.

Скриптовите езици могат да бъдат разделени в две основни категории: динамично изпълними и предварително компилирани. Динамично изпълнимите скриптове четат и изпълняват инструкциите от програмен файл, като обработват един по един функционалните блокове, без да преминават през целия код. Това позволява бързо изпълнение на определени операции, но води до по-ниска производителност в случай на големи и сложни програми. Въпреки това,

предимството е в бързото им внедряване и лесно откриване на грешки по време на разработката.

Предварително компилираните скриптов езици, от друга страна, прочитат и компилират целия код предварително в машинен или междинен формат, което подобрява ефективността по време на изпълнение. Това обаче изисква допълнително време за компилация преди стартиране на изпълнението, но осигурява значително по-добра производителност при големи приложения.

Основните предимства на скриптовите езици включват краткото време за разработка и леснотата при откриването на грешки. Това позволява на разработчиците бързо да тестват нови идеи и да правят корекции на живо, без да се налага да чакат дълго време за компилация. Освен това, скриптовите езици предлагат гъвкавост и са по-лесни за използване при динамични уеб проекти, където промените и актуализациите трябва да бъдат внедрени бързо и ефективно.

2.2.1 HTML (HyperText Markup Language)

HTML (HyperText Markup Language) е основният език за създаване и описание на уеб страници. Той представлява стандарт в Интернет, а неговите правила и спецификации се поддържат и разработват от международната организация W3C (World Wide Web Consortium). В момента последната и най-широко използвана версия на стандарта е HTML5, която въвежда множество нови функционалности и елементи.

HTML използва специални елементи, наречени тагове (HTML tags), които служат за структурирането и форматирането на съдържанието на уеб страниците. HTML елементите са основните компоненти, чрез които се изграждат всички части на една уеб страница, като заглавия, параграфи, списъци, връзки, изображения и други. Повечето HTML елементи са съставени от двойка тагове – отварящ таг, например `<h1>`, и затварящ таг, например `</h1>`, които обграждат съдържанието на елемента.

HTML се използва за определяне на структурата на уеб страниците и за контролиране на това, което браузърът показва на екрана. Основните функции на HTML включват управление на текстовия поток (Flow Control), добавяне на изображения (Images), създаване на хипервръзки (Links), интегриране на аудио и

видео съдържание чрез тагове като <audio> и <video>, вмъкване на външни ресурси чрез <iframe>, създаване на формуляри (Forms) за въвеждане на данни, разделяне на страницата на секции с помощта на <div> и <section>, както и интеграция на външни приложения чрез тагове като <script>.

HTML документите обикновено се създават като текстови файлове и се хостват на уеб сървъри, достъпни в Интернет. Те съдържат текстово съдържание, обградено с HTML тагове, които служат като инструкции за браузърите за начина на визуализиране на съдържанието. Браузърите не показват самите тагове, а само резултата от тяхното интерпретиране – текст, изображения, видеа и други елементи на страницата.

Едно от най-големите предимства на HTML е неговата универсалност – уеб страниците, създадени с HTML, могат да бъдат прегледани на множество различни устройства – от компютри до мобилни телефони и таблети. Това осигурява последователност в дизайна и функционалността на уеб страниците.

HTML код може да бъде написан ръчно в текстови редактори като Notepad, Notepad++, Sublime Text и други. В същото време съществуват и специализирани инструменти като Microsoft FrontPage, Adobe Dreamweaver и NetBeans, които улесняват процеса на създаване на HTML страници, без да изискват детайлно познаване на кода. Тези инструменти предоставят графичен интерфейс и автоматизация, което прави работата по-лесна и достъпна дори за начинаещи потребители.

Семантични елементи в HTML5:

- **Елемент <header>** – представлява заглавие на елемент или група от елементи. Тагът <header> позволява използването на няколко заглавия в един документ.
- **Елемент <nav>** – дефинира навигацията на сайта. Предназначен е за големи блокове от навигационни връзки. Не всички връзки трябва да бъдат поставени в <nav>, а само тези, които са част от главното навигационно меню.
- **Елемент <article>** – представлява самостоятелно съдържание или статия. Съдържанието в <article> трябва да бъде самодостатъчно и способно да се разпространява независимо от останалата част на сайта. Най-често се използва за публикации във форуми, блогове, новинарски статии, коментари и други.

- **Елемент <section>** – тематично групирано съдържание, което обикновено е придружено със заглавие. Този елемент дава възможност за създаване на нова семантична структура в рамките на вече съществуващ елемент.
- **Елемент <aside>** – представя съдържание, което е странично спрямо основното съдържание на страницата. Съдържанието в <aside> се счита за второстепенно и може да бъде пропуснато, ако сайтът се разглежда на устройство с малък екран, като мобилен телефон.
- **Елемент <footer>** – съдържа информация за автора на страницата, навигационни връзки, авторски права върху използваните материали, както и връзки към други (подобни) публикации.

```
<!DOCTYPE html>
<html>

  <head>
    <title>My First Webpage</title>
  </head>

  <body>
    <h1>
      My First Webpage
    </h1>
    <p>This is a paragraph...</p>
  </body>

</html>
```

Фиг.1 Пример за HTML код

2.2.2 CSS (Cascading Style Sheets)

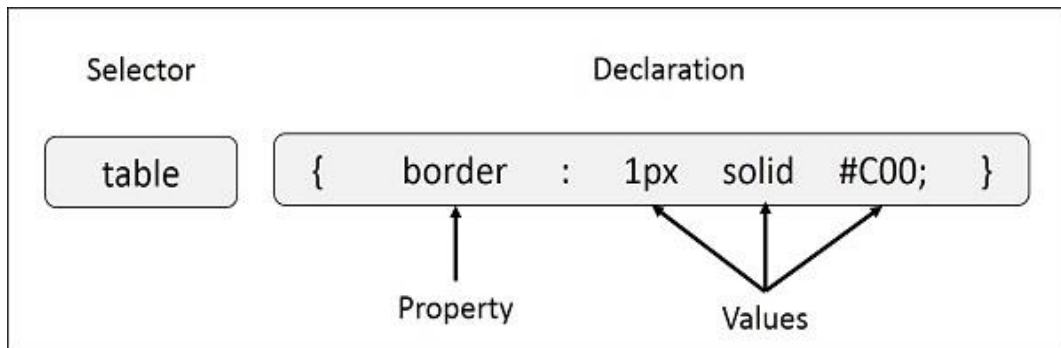
CSS (Cascading Style Sheets) е език, който определя начина, по който изглежда и се представя съдържанието на уеб страниците. Той работи съвместно с HTML или XML, като осигурява визуално форматиране на елементите, като шрифтове, цветове, размери, разположение и други аспекти на дизайна.

Разработката и поддръжката на CSS стандарта се извършва от W3C (World Wide Web Consortium), което гарантира неговата съвместимост и развитие.

Основният принцип на CSS е използването на стилови правила, които определят външния вид на различни елементи. Всяко правило се състои от селектор (елементът, към който се прилага стилът) и декларации, които задават конкретни стилови свойства. Всяка декларация включва свойство (например color, font-size, margin) и стойност (конкретния ефект, който ще бъде приложен, като red, 16px, 10px).

CSS позволява разделянето на съдържанието от визуалното оформление, което улеснява поддръжката на уеб страниците и прави кода по-гъвкав. Чрез него може да се прилагат стилове към отделни елементи, цели секции или дори към цял сайт наведнъж. Освен това CSS предоставя механизми за изграждане на адаптивен дизайн, като се използват различни техники, включително медийни заявки, които позволяват динамично адаптиране към различни устройства и екрани.

CSS не само подобрява естетиката на уеб страниците, но и оптимизира тяхната ефективност, като намалява повторението на код и улеснява управлението на дизайна в мащабни проекти.



Фиг 2. Синтаксис

От лявата страна на свойството се намира селекторът – това е частта от правилото, указваща за кой елемент от документа трябва да се приложат стиловете.

От дясната страна на свойство се намира декларацията, която представлява комбинация от свойството и стойност. Свойството е стилиов атрибут, на който се задава стойност. Всяко свойство има стойност. CSS декларацията винаги завършва с точка и запетая, а групите от декларации са заградени от фигурни скоби.

```

h1 {
    font-family: courier, courier-new, serif;
    font-size: 20pt;
    color: blue;
    border-bottom: 2px solid blue;
}
p {
    font-family: arial, verdana, sans-serif;
    font-size: 12pt;
    color: #6B6BD7;
}
.red_txt {
    color: red;
}

```

Фиг 3. Пример за CSS код:

2.2.3 JavaScript

JavaScript (JS) е един от най-важните и широко използвани езици за програмиране в уеб разработката. Той е създаден през 1995 година от Брендън Айк и се е утвърдил като основен инструмент за създаване на интерактивни и динамични уеб приложения. JavaScript работи директно в уеб браузъра и позволява на разработчиците да добавят анимации, обработка на събития, динамични ефекти и комуникация със сървъра без необходимост от презареждане на страницата.

Една от най-мощните характеристики на JavaScript е асинхронното програмиране. Благодарение на механизми като Event Loop, Callbacks, Promises и Async/Await, езикът позволява ефективна работа със заявки към сървъра, обработка на файлове и други задачи, които се изпълняват без да блокират главния изпълнителен поток.

JavaScript има пряк достъп до Document Object Model (DOM), което позволява на разработчиците динамично да променят съдържанието, структурата и стила на HTML страницата. Това е основната причина, поради която JS е толкова популярен в фронтенд разработката.

JavaScript е неизменна част от уеб разработката, като осигурява интерактивност, динамика и комуникация със сървъра. Неговата гъвкавост, асинхронност и богата екосистема от инструменти го правят един от най-мощните езици за създаване на уеб и мобилни приложения.

С непрекъснатото развитие на нови библиотеки и технологии, JavaScript остава ключов инструмент за всеки разработчик, който иска да изгражда модерни и високоефективни уеб решения.

2.2.4 C#

C# е мощен и широко използван програмен език, създаден от Microsoft през 2000 година като част от .NET платформата. Той е проектиран с идеята за сигурност, бързина и гъвкавост, което го прави един от предпочитаните езици за разработка на софтуер за Windows и други платформи. Благодарение на своята универсалност, C# се използва за изграждане на разнообразни приложения – от уеб и настолни приложения до мобилни приложения и видеоигри.

Една от най-отличителните характеристики на C# е, че той е обектно-ориентиран език. Това означава, че програмистите могат да изграждат сложни софтуерни системи, като използват класове и обекти, които осигуряват логическа структура на кода. Също така, езикът поддържа наследяване и полиморфизъм, които позволяват повторна употреба на кода и по-ефективно управление на сложността в големи проекти.

C# е строго типизиран, което означава, че всяка променлива и обект имат точно определен тип, който не може да бъде променян по време на изпълнението. Това налага допълнителна сигурност и намалява възможността за грешки, като гарантира стабилността на програмите. Освен това езикът разполага с вградени механизми за обработка на изключения, които осигуряват ефективно управление на грешките по време на работа на приложението, улеснявайки неговата поддръжка и подобрявайки надеждността му.

Един от най-големите плюсове на C# е, че управлението на паметта е автоматизирано. Вместо програмистите да се грижат ръчно за освобождаването на памет, .NET платформата предоставя механизъм за автоматично управление на паметта чрез garbage collection. Това позволява разработчиците да се фокусират върху самата логика на приложението, без да се тревожат за изтичане на памет или грешки, свързани с нейното управление.

Освен това, C# разполага с поддръжка за многонишково програмиране, което означава, че може да обработва множество задачи едновременно. Това е

изключително полезно за изграждането на съвременни приложения, които трябва да работят бързо и ефективно, особено в среда с паралелни изчисления и конкурентни операции.

Благодарение на тези характеристики, C# се е наложил като един от най-гъвкавите и надеждни езици за програмиране, подходящ както за начинаещи, така и за опитни разработчици, които търсят стабилна и мощна среда за разработка на софтуер.

2.2.5 ASP.NET Core

ASP.NET Core е усъвършенствана веб платформа, разработена като допълнение към езика C#. Тя представлява едно от най-гъвкавите и високопроизводителни решения за изграждане на веб приложения, като е проектирана от нулата с фокус върху ефективност, разширяемост и независимост от конкретна операционна система. Тази технология позволява създаването на мощни веб решения, които могат да работят на различни платформи, включително Windows, Linux и macOS.

ASP.NET Core е напълно крос-платформен, което означава, че разработчиците не са ограничени до Microsoft екосистемата. Благодарение на .NET Core, приложенията могат да бъдат стартирани и хоствани в различни среди, включително облачни услуги и контейнери.

Архитектурата на платформата е модулна, което позволява на програмистите да използват само тези компоненти, които са необходими за конкретния проект. Това минимизира излишните зависимости и прави приложенията по-ефективни и лесни за управление.

Една от ключовите концепции в ASP.NET Core е Middleware – архитектурен подход, който позволява добавяне на допълнителна функционалност към веб приложенията чрез независими софтуерни компоненти. Middleware може да обработва заявки и отговори, да извършва аутентикация, авторизация, логване, кеширане, компресиране на данни и много други операции. Благодарение на тази концепция, разработчиците могат лесно да изграждат сложни и мощни веб решения, като комбинират различни Middleware компоненти.

ASP.NET Core е напълно отворен код и се поддържа в GitHub. Това предоставя възможност на програмистите не само да използват и модифицират кода, но и да допринасят за неговото развитие. Като проект с активна общност и подкрепа от Microsoft, ASP.NET Core се обновява непрекъснато, което го прави надеждна и модерна платформа за уеб разработка.

Платформата предлага вградена интеграция с облачни услуги, като Microsoft Azure, което я прави подходяща за създаване на мащабируеми и гъвкави уеб приложения. Това означава, че разработчиците могат лесно да разгръщат своите приложения в облака, да използват автоматично управление на ресурсите и да се възползват от надеждността на облачните среди.

ASP.NET Core използва MVC (Model-View-Controller) – добре установен модел за разделяне на логиката на приложението на три отделни слоя:

- **Модел** – отговаря за данните и бизнес логиката.
- **Изглед** – дефинира начина, по който информацията се визуализира пред потребителя.
- **Контролер** – управлява заявките от клиента и ги насочва към съответните обработващи компоненти.

Тази структура подобрява организацията на кода, улеснява поддръжката и позволява по-добра мащабируемост на уеб приложенията.

T Core разполага с вградена поддръжка за Web API, което позволява създаването на RESTful уеб услуги. Това е особено важно за изграждането на мащабируеми приложения, които трябва да комуникират с различни клиенти – уеб приложения, мобилни устройства и IoT платформи. Благодарение на своята мултиплатформена поддръжка, модулна архитектура, Middleware концепция и интеграция с облачни технологии, тя се е наложила като едно от най-добрите решения за уеб разработка.

Разработчиците, които изберат ASP.NET Core, получават сигурност, висока производителност и мащабируемост, като в същото време разполагат с богата екосистема от инструменти и библиотеки за създаване на иновативни уеб решения.

на Web API, програмистите могат лесно да разработват и разгръщат гъвкави и достъпни уеб услуги.

ASP.NET Core предлага вградена поддръжка за сигурност, включително Identity Framework – система за управление на потребители, пароли, роли и

разрешения. Това е особено важно за приложения, които обработват чувствителни данни, като лична информация или финансови операции.

ASP.NET Core е модерна, ефективна и високопроизводителна платформа, която предоставя гъвкавост и мощни инструменти за разработка на уеб приложения.

2.3 Модели и езици за база данни и СУБД

Базата данни представлява организирана структура, която съдържа свързани помежду си данни в конкретна предметна област. Данните се подреждат по начин, който позволява ефективно извличане и обработка чрез компютърни системи. В основата на всяка база данни стоят записи, състоящи се от отделни елементи, които при заявка се превръщат в информация, използвана за вземане на решения.

За да се управляват базите от данни, се използват специализирани софтуерни решения, наречени Системи за управление на бази данни (СУБД). Те дават възможност за организирано съхраняване, достъп, манипулация и защита на данните.

От гледна точка на потребителя, ефективната система за управление на бази данни трябва да притежава няколко ключови свойства:

- Устойчивост на данните – Информацията трябва да се запазва дългосрочно, независимо от текущите процеси, като достъпът до нея трябва да бъде бърз и ефективен, независимо от обема ѝ.
- Програмен интерфейс – СУБД трябва да разполага с мощен език за заявки, който позволява на потребителите и приложенията да комуникират с базата данни.
- Управление на транзакции – Транзакциите са групи операции, които се изпълняват като един цялостен процес. СУБД гарантира, че транзакциите са атомарни (изпълняват се напълно или изобщо не се изпълняват), изолирани (не влияят на други паралелни транзакции) и устойчиви (резултатите от тях се запазват дори при сринове).
- Възстановяване при грешки – Системата трябва да разполага с механизми за защита на данните и възстановяване в случай на сринове или неочаквани грешки.

Освен осигуряване на надеждност и ефективност, една СУБД предоставя и конкретни инструменти за работа с базите от данни:

- Създаване и управление на бази от данни – Потребителят може да дефинира логическата структура на базата чрез Език за дефиниция на данните (DDL).
- Извличане и обработка на информация – Данните се търсят, филтрират и модифицират чрез Език за манипулация на данни (DML).
- Дългосрочно съхранение – СУБД трябва да осигурява оптимизирано съхранение и достъп до големи обеми информация.
- Едновременен достъп – Много потребители могат да работят с данните паралелно, без да се компрометира тяхната цялост и последователност.

Базите данни са в основата на съвременните информационни системи, като тяхното управление изисква специализирани софтуерни решения – СУБД. Те осигуряват надеждност, сигурност, ефективност и възможност за многопотребителска работа, като същевременно предоставят инструменти за лесно създаване, манипулиране и защита на данните.

2.3.1.1 Релационен модел

В релационните бази данни информацията се съхранява в таблици, като всяка таблица има уникално име. Всяка колона (атрибут) също има уникално име, а всеки ред в таблицата представлява уникален запис.

- Таблиците са свързани помежду си чрез връзки (релации), които се основават на ключове. Основните типове връзки между таблиците са:
- Един към много (one-to-many) – една стойност от едната таблица може да съответства на много стойности от другата.
- Много към много (many-to-many) – множество записи в едната таблица могат да бъдат свързани с множество записи в другата.
- Един към един (one-to-one) – всеки запис от едната таблица съответства на точно един запис в другата.

За да се избегне дублирането на информация и да се осигури по-добра организация, се използва нормализация. Това е процес на разделяне на данните в по-малки таблици, което подобрява ефективността, надеждността и бързината на операциите.

Релационните бази данни се управляват чрез SQL (Structured Query Language) – мощен език, който позволява извличане, добавяне, актуализиране и изтриване на данни чрез команди като SELECT, INSERT, UPDATE и DELETE.

Заради гъвкавостта, надеждността и мащабируемостта си, релационните бази данни намират широко приложение в различни области като финанси, търговия, управление на персонал и много други.

2.3.1.2 Нерелационен модел

Нерелационният модел, известен още като NoSQL (Not Only SQL), представлява начин за организиране, съхранение и обработка на данни, който се различава от традиционните релационни бази данни. Докато релационните бази използват таблици с предварително дефинирана схема, състояща се от редове и колони, нерелационните бази предлагат по-гъвкави структури за съхранение. Те не изискват строго дефинирана схема и позволяват по-лесно адаптиране към различни типове данни. Това ги прави особено подходящи за работа с големи, динамично променящи се и неструктурирани данни, които често срещаме в съвременните уеб приложения, анализи на големи данни, системи за препоръки и Internet of Things (IoT).

Основното предимство на нерелационните бази данни е тяхната способност да се мащабират хоризонтално. Докато релационните бази често изискват мощен хардуер за обработка на нарастващи обеми информация (вертикално мащабиране), NoSQL базите позволяват разпределение на данните върху множество сървъри или клъстери, което подобрява производителността и устойчивостта на системата. Това ги прави предпочитани за приложения, които трябва да обработват огромни количества информация в реално време, като социални мрежи, облачни услуги и онлайн търговия.

2.3.2 SQL (Structured Query Language)

SQL (Structured Query Language) е език, предназначен за работа с релационни бази данни. Той се състои от набор от команди, които позволяват на потребителите да управляват данните в базата. Основните команди включват

SELECT за извличане на данни, INSERT за добавяне на нови записи, UPDATE за актуализиране на съществуващи записи и DELETE за премахване на данни. Тези команди могат да се изпълняват както чрез специализирани програми, така и чрез интерактивни среди за управление на бази данни.

Релационните бази данни, с които работи SQL, са организирани в таблици, състоящи се от редове и колони. Всяка колона представлява определен атрибут или поле, а всеки ред съдържа конкретни данни, наречени записи. За да се осигури целостта на данните, таблиците обикновено имат първичен ключ (Primary Key), който гарантира уникалността на всеки запис. Освен това таблиците могат да бъдат свързани помежду си чрез външни ключове (Foreign Keys), което позволява изграждане на сложни релационни структури.

SQL предоставя и множество функции за обработка на данни. Чрез вградените математически операции могат да се извършват изчисления директно в заявките. Езикът също така поддържа конкатенация на низове, форматиране на дати, агрегатни функции като SUM, AVG, COUNT, MAX и MIN, които се използват за анализ и обработка на големи обеми данни. Освен това SQL дава възможност за сортиране и филтриране на информация чрез ORDER BY и WHERE клаузи.

Едно от най-големите предимства на SQL е способността му да комбинира данни от множество таблици чрез JOIN операторите. Например, INNER JOIN извлича само съвпадащите записи между две таблици, докато LEFT JOIN връща всички записи от лявата таблица и съвпадащите записи от дясната. Това позволява на разработчиците да извличат и анализират данни по сложни начини, които са необходими за реални бизнес приложения.

SQL е изключително важен за разработката на софтуер, който използва релационни бази данни, като уеб приложения, корпоративни системи, анализ на данни и финансови платформи. Езикът се поддържа от много различни системи за управление на бази данни (СУБД), включително MySQL, PostgreSQL, Oracle и Microsoft SQL Server. Благодарение на своята универсалност и стандартизация, SQL може да се използва на различни операционни системи и платформи.

В заключение, SQL е мощен инструмент за работа с данни, който осигурява гъвкавост, надеждност и ефективност. Той позволява създаване, манипулиране и анализ на релационни бази данни, като същевременно предоставя механизми за управление на достъпа и сигурността на информацията.

2.3.3 СУБД (Система за управление на бази данни)

Системата за управление на бази данни (СУБД) е софтуер, който улеснява създаването, поддръжката и управлението на бази данни. Тя предоставя ефективен начин за съхранение, обработка и извличане на информация във формат, който е удобен за използване от потребителите.

СУБД играе ключова роля в управлението на данни в различни организации, като позволява на потребителите да въвеждат, редактират и изтриват информация, както и да извършват търсения и заявки в базите данни. Освен това тя осигурява допълнителни функционалности като транзакции, репликации и различни методи за обработка на данни, което я прави мощен инструмент за работа с големи обеми информация.

Значението на СУБД е особено голямо в бизнеса и технологичните индустрии, тъй като без нея управлението на големи количества данни би било значително по-трудно и неефективно. Повечето популярни системи за управление на бази данни са базирани на релационния модел, като MySQL, PostgreSQL и Oracle. В допълнение към тях съществуват и нерелационни (NoSQL) бази данни, които се използват в специфични случаи, когато традиционните релационни структури не са достатъчно гъвкави.

Част 3. Реализация на план за изготвяне на уеб-страница за конна база

3.1 Концепция

Проектът „Конна база“ е уебсайт, предназначен за разглеждане и резервиране на коне. Потребителите могат да търсят и избират коне по различни критерии, както и да правят резервации за определен период от време. Системата също така предоставя възможност за преглед на предишни резервации, което улеснява управлението на направените заявки.

3.2 Design pattern

Дизайнерският шаблон (Design Pattern) представлява утвърдено и повторяемо решение за често срещани проблеми при проектирането на софтуер. Тези шаблони са създадени, за да предлагат ефективни и проверени подходи, които разработчиците могат да използват в своите проекти.

Всеки дизайнерски шаблон включва две основни части – проблема, който решава, и препоръчителния начин за неговото преодоляване. Когато разработчикът се сблъска с даден проблем, който съответства на някой от тези шаблони, той може да приложи предлаганото решение, което ще му помогне да избере по-добър и по-ефективен подход за реализация.

3.3 MVC design pattern

MVC (Model-View-Controller) е софтуерен дизайнерски шаблон, който разделя приложението на три основни компонента: модел, изглед и контролер. Тази архитектура осигурява гъвкавост и улеснява поддръжката, тъй като всяка част може да бъде разработвана и модифицирана независимо от останалите.

Моделът отговаря за управлението на данните и логиката на приложението. Той обработва входните данни, валидира ги и предоставя информация на изгледа. Изгледът е отговорен за визуализацията и представя на потребителя обработените от модела данни. Контролерът служи като посредник между модела и изгледа, като обработва потребителските заявки и изпраща подходящите данни към изгледа.

Тъй като компонентите в MVC са отделени, те могат да се разработват и тестват независимо. Това прави този подход особено полезен за големи софтуерни проекти, където различни екипи могат да работят паралелно върху отделни части от приложението. MVC е широко използван в уеб разработката, но намира приложение и в други видове софтуер.

Прилагането на този шаблон прави приложенията по-гъвкави, лесни за поддръжка и разширяване. Освен това, разделянето на компонентите позволява по-бърза разработка, тъй като програмистите могат да работят независимо един от друг върху различните части на системата.

3.4 Модели в приложението „Конна база“

Моделът в архитектурата MVC е ключов компонент, който осигурява функционирането на приложението на ниво данни и бизнес логика. Той не се занимава с представянето на данните на потребителя, а с тяхното съхраняване, обработка и манипулация, като по този начин отделя логиката за управлението на данните от визуализацията и потребителския интерфейс. Във всяко приложение, което използва MVC архитектура, моделът е отговорен за всички задачи, които се отнасят до данни и операции, свързани с тях.

Основната функция на модела е да съхранява данни, които могат да бъдат използвани от различни части на приложението. Например, в една система за управление на потребители, моделът ще съдържа данни като имена, адреси, електронни пощи, пароли и други потребителски данни. Тези данни могат да бъдат съхранявани в база данни, файлова система или дори в паметта, в зависимост от спецификата на приложението.

Когато моделът претърпява промени в данни, той уведомява контролера или изгледа (View), че данните са обновени и трябва да бъдат визуализирани или обработени по нов начин. В MVC архитектурата моделът е изолирана единица, което позволява по-лесно тестване и поддръжка на бизнес логиката, тъй като той е независим от представянето и взаимодействията с потребителя.

3.4.1 Потребители (AspNetUsers)

Потребителите, които ASP.NET създава съдържат следните полета:

- Id
- UserName
- NormalizedUserName
- Email
- NormalizedEmail
- EmailConfirmed
- PasswordHash
- SecurityStamp
- ConcurrencyStamp
- PhoneNumber
- PhoneNumberConfirmed
- TwoFactorEnabled
- LockoutEnd
- LockoutEnabled
- AccessFailedCount
- Discriminator

За да е пълна задачата, таблицата трябва да има и полетата:

- FirstName
- MiddleName
- LastName
- IsActive

За добавянето на тези полета се прави класа User.cs. Той наследява IdentityUser, който е класа за потребители на ASP.NET и финализира потребителя.

```
public class User : IdentityUser
{
    public string FirstName { get; set; }
    public string MiddleName { get; set; }
    public string LastName { get; set; }
    public bool IsActive { get; set; }
}
```

Фиг 4. Модела за потребителите

3.4.2 Кон

Всеки кон се характеризира с:

- Номер
- Порода
- Година на раждане
- Пол
- Височина
- Цена (на час)
- Снимки

```
public class Horse
{
    public int Id { get; set; }

    [Required, Display(Name = "Номер")]
    public int Number { get; set; }

    public int BreedId { get; set; }

    public Breed? Breed { get; set; }

    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:d M yyyy}")]
    [Required, Display(Name = "Година на раждане")]
    public DateTime BirthYear { get; set; }

    [Required, Display(Name = "Пол")]
    public string Gender { get; set; }

    [Required, Display(Name = "Височина")]
    public int Height { get; set; }

    [DataType(DataType.Currency)]
    [Required, Display(Name = "Цена")]
    public double Price { get; set; }

    [Display(Name = "Снимки")]
    public string? PhotoPath { get; set; }
}
```

Фиг 5. Модела за конете

3.4.3 Порода

Всяка порода се характеризира с:

- Име
- Снимка

```
public class Breed
{
    public int Id { get; set; }

    [Required, Display(Name = "Название")]
    public string Name { get; set; }

    [DataType(DataType.ImageUrl)]
    [Required, Display(Name = "Връзка")]
    public string Url { get; set; }

    ICollection<Horse> Horses { get; set; }
}
```

Фиг. 6 Модела за породите

3.4.4 Резервации

Всяка резервация се характеризира с:

- Потребител
- Кон
- Дата и час на вземане
- Дата и час на връщане
- Дължима сума

```
public class Reservation
{
    public int Id { get; set; }

    [Required, Display(Name = "Потребител")]
    [ForeignKey("User")]
    public string UserId { get; set; }
    public User User { get; set; }

    [Required, Display(Name = "Кон")]
    public Horse Horse { get; set; }

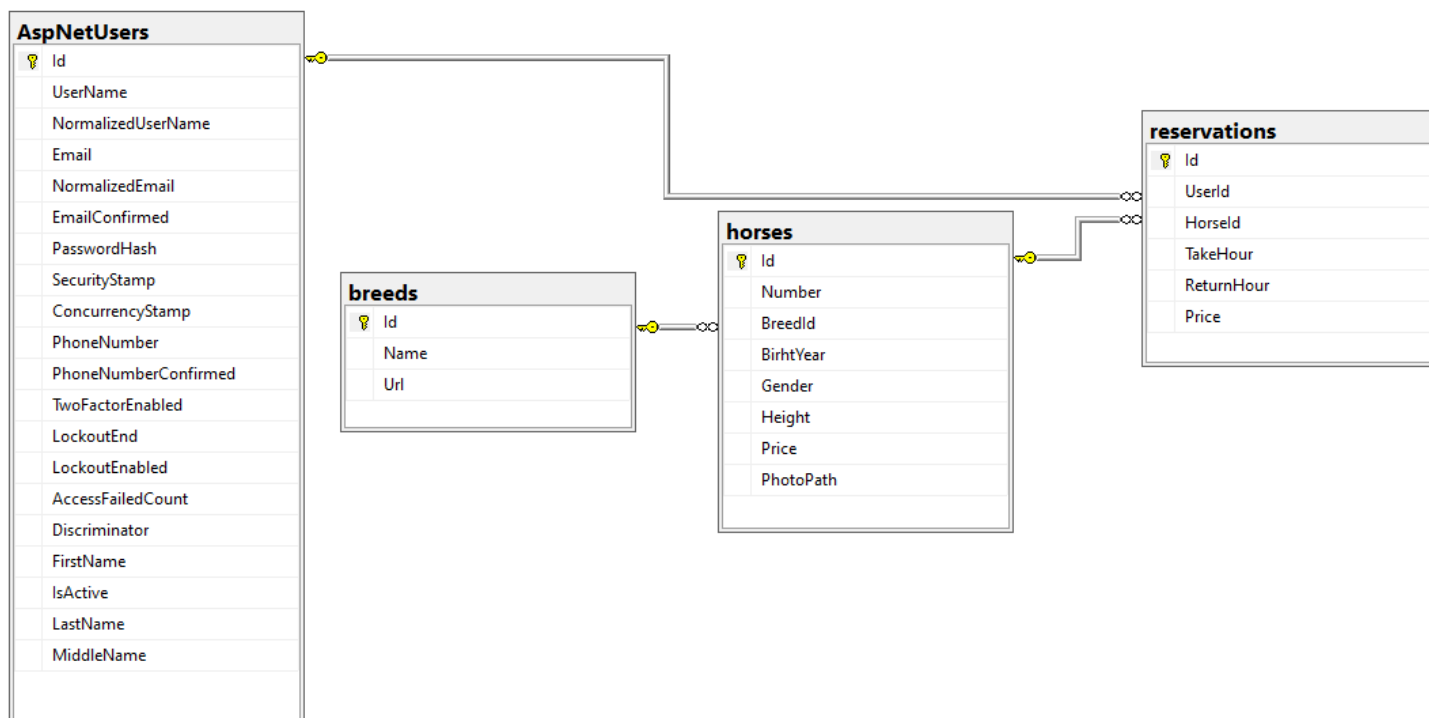
    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:d/M/yyyy - H:mm}")]
    [Required, Display(Name = "Дата и час на вземане")]
    public DateTime TakeHour { get; set; }

    [DataType(DataType.Date)]
    [DisplayFormat(DataFormatString = "{0:d/M/yyyy - H:mm}")]
    [Required, Display(Name = "Дата и час на връщане")]
    public DateTime ReturnHour { get; set; }

    [DataType(DataType.Currency)]
    [Required, Display(Name = "Цена")]
    public double Price { get; set; }
}
```

Фиг. 7 Модела за резервациите

3.4.5 Диаграма на базата данни



3.5 Контролери

Контролерът в архитектурата MVC (Model-View-Controller) е един от основните компоненти, който отговаря за управлението на взаимодействието между потребителя и приложението. Той приема заявки от потребителя, обработва ги и връща съответния отговор.

Основната роля на контролера е да действа като посредник между модела (който съдържа и обработва данните) и изгледа (който отговаря за визуализацията). Когато потребителят извърши дадено действие, например натисне бутон или въведе информация във формуляр, заявката се изпраща към контролера. Той анализира заявката, определя какви действия трябва да се предприемат, и извиква съответните методи от модела за обработка на данните.

След като моделът обработи заявката и предостави резултатите, контролерът решава как тази информация да бъде представена на потребителя. Той избира подходящия изглед и му подава необходимите данни за визуализация.

Контролерът може също така да съдържа логика за валидация на входните данни, управление на потребителски сесии и обработка на грешки. По този начин той гарантира правилната работа на приложението, като координира действията между модела и изгледа.

3.5.1 HomeController

HomeController в ASP.NET Core MVC обработва заявките към началната страница (Index). Той използва ILogger за логване на събития. Това е основен контролер, който се използва за стартиране на уеб приложението.

```
using HorseBase.Models;
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;

namespace HorseBase.Controllers
{
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        public IActionResult Index()
        {
            return View();
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
        }
    }
}
```

Фиг. 8 Класа “HomeCotroller”

3.5.2 HorseController

HorseController управлява заявките, свързани с конете, като осигурява функционалности за показване на списък с коне, детайлна информация за конкретен кон, създаване на нов запис, редактиране на съществуващи данни и изтриване на коне от базата. Контролерът също така позволява филтриране на конете по различни критерии, като порода, година на раждане, пол, височина и цена, за да улесни търсенето. Освен това, той обработва качването и управлението на снимки, като ги съхранява в локалната файлова система и осигурява правилното им асоцииране с конкретните записи в базата данни.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.EntityFrameworkCore;
using HorseBase.Data;
using HorseBase.Models;
using System.Linq;
using System.Threading.Tasks;
using Newtonsoft.Json;

namespace HorseBase.Controllers
{
    public class HorseController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly IWebHostEnvironment _webHostEnvironment;

        public HorseController(ApplicationDbContext context, IWebHostEnvironment webHostEnvironment)
        {
            _context = context;
            _webHostEnvironment = webHostEnvironment;
        }
    }
}
```

Фиг. 9 Част от класа "HorseController"

3.5.2.1 Index

Извлича списък с коне и прилага филтри като порода, година на раждане, пол, височина и цена. Данните се изпращат към изгледа.

```
public async Task<IActionResult> Index(
    string breed,
    int? birthYear,
    string gender,
    int? height,
    int? price)
{
    // Start with the base query
    var horsesQuery = _context.horses.Include(h => h.Breed).AsQueryable();

    // Apply filters if they are provided
    if (!string.IsNullOrEmpty(breed))
    {
        horsesQuery = horsesQuery.Where(h => h.Breed.Name == breed);
    }

    if (birthYear.HasValue)
    {
        horsesQuery = horsesQuery.Where(h => h.BirthYear.Year == birthYear.Value);
    }

    if (!string.IsNullOrEmpty(gender))
    {
        horsesQuery = horsesQuery.Where(h => h.Gender == gender);
    }

    if (height.HasValue)
    {
        horsesQuery = horsesQuery.Where(h => h.Height <= height.Value);
    }

    if (price.HasValue)
    {
        horsesQuery = horsesQuery.Where(h => h.Price <= price.Value);
    }

    // Pass filtered data to the view
    var horses = await horsesQuery.ToListAsync();

    // Populate ViewBag for filter options
    ViewBag.Breeds = new SelectList(_context.breeds, "Name", "Name");

    return View(horses);
}
```

Фиг. 10 метода "Index"

3.5.2.2 Details

Показва детайлна информация за кон по дадено id.

```
public async Task<IActionResult> Details(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var horse = await _context.horses
        .Include(h => h.Breed)
        .FirstOrDefaultAsync(m => m.Id == id);
    if (horse == null)
    {
        return NotFound();
    }

    return View(horse);
}
```

Фиг. 11 Метода “Details”

3.5.2.3 Create (GET)

Зарежда формата за създаване на нов кон с възможност за избор на порода.

```
public IActionResult Create()
{
    ViewData["BreedId"] = new SelectList(_context.breeds, "Id", "Name");
    return View();
}
```

Фиг. 12 – метода “Create (GET)”

3.5.2.4 Create (POST)

Създава нов кон, като обработва качените снимки и ги съхранява.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Id,Number,BreedId,BirthYear,Gender,Height,Price,PhotoPath")] Horse horse, IFormFile[] photos)
{
    if (photos != null && photos.Length > 0)
    {
        var imagePaths = new List<string>();

        foreach (var photo in photos)
        {
            if (photo.Length > 0)
            {
                var fileName = Path.GetFileNameWithoutExtension(photo.FileName);
                var extension = Path.GetExtension(photo.FileName);
                var filePath = Path.Combine(_webHostEnvironment.WebRootPath, "images", fileName + extension);

                using (var stream = new FileStream(filePath, FileMode.Create))
                {
                    await photo.CopyToAsync(stream);
                }
                imagePaths.Add("/images/" + fileName + extension);
            }
        }
        horse.PhotoPath = string.Join(",", imagePaths);
    }

    _context.Add(horse);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

Фиг. 13 Метода “Create (POST)”

3.5.2.5 Edit (GET)

Зарежда формата за редактиране на съществуващ кон и показва текущите му СНИМКИ.

```
[HttpGet]
public async Task<IActionResult> Edit(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var horse = await _context.horses.FindAsync(id);
    if (horse == null)
    {
        return NotFound();
    }

    // Pass the existing photos to the view
    ViewBag.ExistingPhotos = string.IsNullOrEmpty(horse.PhotoPath) ? new List<string>() : horse.PhotoPath.Split(',').ToList();

    ViewData["BreedId"] = new SelectList(_context.breeds, "Id", "Name", horse.BreedId);
    return View(horse);
}
```

Фиг. 14 Метода “Edit (GET)”

3.5.2.6 Edit (POST)

Обновява информацията за кон, като позволява добавяне на нови снимки и запазване на старите.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, IFormFile[] photos, [Bind("Id,Number,BreedId,BirthYear,Gender,Height,Price,PhotoPath")] Horse horse)
{
    if (id != horse.Id)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            // Handle photo uploads
            if (photos != null && photos.Length > 0)
            {
                var photoPaths = new List<string>();
                foreach (var photo in photos)
                {
                    if (photo.Length > 0)
                    {
                        var filePath = Path.Combine("wwwroot/images", Guid.NewGuid().ToString() + Path.GetExtension(photo.FileName));
                        using (var stream = new FileStream(filePath, FileMode.Create))
                        {
                            await photo.CopyToAsync(stream);
                        }
                        photoPaths.Add("/images/" + Path.GetFileName(filePath));
                    }
                }

                // Combine new photos with existing ones
                if (!string.IsNullOrEmpty(horse.PhotoPath))
                {
                    photoPaths.AddRange(horse.PhotoPath.Split(', '));
                }

                horse.PhotoPath = string.Join(",", photoPaths.Distinct());
            }

            _context.Update(horse);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!HorseExists(horse.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
        return RedirectToAction(nameof(Index));
    }

    ViewData["BreedId"] = new SelectList(_context.breeds, "Id", "Name", horse.BreedId);
    return View(horse);
}
```

Фиг. 15 Метода “Edit (POST)”

3.5.2.7 HorseExists

Проверява дали даден кон съществува в базата.

```
private bool HorseExists(int id)
{
    return _context.horses.Any(e => e.Id == id);
}
```

Фиг. 16 Метода “HorseExists”

3.5.2.8 Delete (GET)

Зарежда потвърдителната страница за изтриване на кон.

```
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var horse = await _context.horses
        .Include(h => h.Breed)
        .FirstOrDefaultAsync(m => m.Id == id);
    if (horse == null)
    {
        return NotFound();
    }

    return View(horse);
}
```

Фиг. 17 Метода “Delete (GET)”

3.5.2.9 DeleteConfirmed (POST)

Изтрива кон от базата данни и пренасочва към списъка.

```
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var horse = await _context.horses.FindAsync(id);
    if (horse != null)
    {
        _context.horses.Remove(horse);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

Фиг. 18 Метода “DeleteConfirmed (POST)”

3.5.2.10 GetAllHorses

Връща списък с всички коне в JSON формат, като проверява дали потребителят е логнат.

```
public string GetAllHorses()
{
    // Check if the user is logged in and has the "User" or "Admin" role
    bool isLogged = User.IsInRole("User") || User.IsInRole("Admin");

    // Fetch the list of horses with their associated breed information
    var horses = _context.horses.Include(x => x.Breed).ToList();

    // Create an anonymous object to hold both the horses and the login status
    var result = new
    {
        Horses = horses,
        IsLogged = isLogged
    };

    // Serialize the result to JSON and return it
    return JsonConvert.SerializeObject(result);
}
```

Фиг. 19 Метода “GetAllHorses”

3.5.3 UsersController

UsersController управлява регистрацията, влизането, излизането и управлението на потребителите. Той позволява създаване на нови акаунти, проверка на идентификационни данни, активиране/деактивиране на потребители, търсене и показване на списък с потребители, както и управление на техните резервации.

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using HorseBase.Models.ViewModels.User;
using HorseBase.Models;
using HorseBase.Data;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;

namespace HorseBase.Controllers
{
    public class UsersController : Controller
    {
        private readonly UserManager<User> _userManager;
        private readonly SignInManager<User> _signInManager;
        private readonly ApplicationDbContext _context;
        public UsersController(UserManager<User> userManager, SignInManager<User> signInManager, ApplicationDbContext context)
        {
            _userManager = userManager;
            _signInManager = signInManager;
            _context = context;
        }
    }
}
```

Фиг. 20 Част от класа “UserController”

3.5.3.1 Register (GET)

Връща изгледа за регистрация.

```
[HttpGet]
public IActionResult Register()
{
    return View();
}
```

Фиг. 21 Метода “Register (GET)”

3.5.3.2 Register (POST)

Проверява дали въведените данни са валидни, създава нов потребител и го добавя към ролята "User". Ако регистрацията е успешна, автоматично вписва потребителя и го препраща към началната страница.

```
[HttpPost]
public async Task<IActionResult> Register(UserRegister userRegister)
{
    if (ModelState.IsValid)
    {
        User user = new User()
        {
            UserName = userRegister.UserName,
            Email = userRegister.Email,
            FirstName = userRegister.FirstName,
            MiddleName = userRegister.MiddleName,
            LastName = userRegister.LastName,
            IsActive = true
        };
        var result = await _userManager.CreateAsync(user, userRegister.Password);

        if (result.Succeeded)
        {
            await _userManager.AddToRoleAsync(user, "User");
            await _signInManager.SignInAsync(user, isPersistent: false);

            return RedirectToAction("Index", "Home");
        }
        ViewData["error"] = "Вече има потребител с тези данни.";
    }
    return View(userRegister);
}
```

Фиг. 22 Метода "Register (POST)"

3.5.3.3 LogIn (GET)

Връща изгледа за вход.

```
[HttpGet]
public IActionResult LogIn()
{
    return View();
}
```

Фиг. 23 Метода "LogIn (GET)"

3.5.3.4 LogIn (POST)

Проверява дали потребителят съществува и дали паролата е вярна. Ако потребителят е активен, го вписва, иначе го препраща към страница за отказан достъп.

```
[HttpPost]
public async Task<IActionResult> LogIn(UserLogin logInRequest)
{
    if (ModelState.IsValid)
    {
        var user = await _userManager.FindByEmailAsync(logInRequest.Email);

        if (user != null)
        {
            var passwordCheck = await _userManager.CheckPasswordAsync(user, logInRequest.Password);

            if (passwordCheck)
            {
                if (!user.IsActive)
                {
                    // If the user is inactive, redirect to the No Access page
                    return RedirectToAction("Banned", "Home");
                }

                // Otherwise, sign in the user
                await _signInManager.SignInAsync(user, isPersistent: false);
                return RedirectToAction("Index", "Home");
            }
        }
    }
    return View(logInRequest);
}
```

Фиг. 24 Метода “LogIn (POST)”

3.5.3.5 List (GET)

Извлича всички потребители от базата, включително техните роли, и ги изпраща към изгледа.

```
[HttpGet]
public async Task<IActionResult> List()
{
    // Fetch all users from the UserManager
    var users = _userManager.Users.ToList();

    // Create a list of UserViewModel to pass to the view
    var userViewModels = new List<UserViewModel>();

    foreach (var user in users)
    {
        // Get the roles for the user
        var roles = await _userManager.GetRolesAsync(user);

        // Create a UserViewModel for each user
        var userViewModel = new UserViewModel
        {
            Id = user.Id,
            UserName = user.UserName,
            Email = user.Email,
            FirstName = user.FirstName,
            MiddleName = user.MiddleName,
            LastName = user.LastName,
            IsActive = user.IsActive,
            Roles = roles.ToList()
        };

        userViewModels.Add(userViewModel);
    }

    // Pass the list of UserViewModel to the view
    return View(userViewModels);
}
```

Фиг. 25 Метода “List (GET)”

3.5.3.6 List(POST)

Използва подаден текст за търсене, за да филтрира списъка с потребители по потребителско име, имейл или име.

```
[HttpPost]
public async Task<IActionResult> List(string? searchInput)
{
    // Fetch all users from the UserManager
    var users = _userManager.Users.ToList();
    if (!string.IsNullOrEmpty(searchInput))
    {
        users = users.Where(x =>
            x.UserName.Contains(searchInput, StringComparison.OrdinalIgnoreCase) ||
            x.Email.Contains(searchInput, StringComparison.OrdinalIgnoreCase) ||
            x.FirstName.Contains(searchInput, StringComparison.OrdinalIgnoreCase) ||
            x.MiddleName.Contains(searchInput, StringComparison.OrdinalIgnoreCase) ||
            x.LastName.Contains(searchInput, StringComparison.OrdinalIgnoreCase)
        ).ToList();
    }

    // Create a list of UserViewModel to pass to the view
    var userViewModels = new List<UserViewModel>();

    foreach (var user in users)
    {
        // Get the roles for the user
        var roles = await _userManager.GetRolesAsync(user);

        // Create a UserViewModel for each user
        var userViewModel = new UserViewModel
        {
            Id = user.Id,
            UserName = user.UserName,
            Email = user.Email,
            FirstName = user.FirstName,
            MiddleName = user.MiddleName,
            LastName = user.LastName,
            IsActive = user.IsActive,
            Roles = roles.ToList()
        };

        userViewModels.Add(userViewModel);
    }

    // Pass the list of UserViewModel to the view
    return View(userViewModels);
}
```

Фиг. 26 Метода “List (POST)”

3.5.3.7 UpdateStatus (POST)

Обновява статуса на потребителите (активен/неактивен) въз основа на маркираните полета в списъка.

```
[HttpPost]
public async Task<IActionResult> UpdateStatus(string[] IsActive)
{
    foreach(var user in _context.users)
    {
        if (IsActive.Contains(user.Id))
        {
            user.IsActive = true;
            await _userManager.UpdateAsync(user);
        }
        else
        {
            user.IsActive = false;
            await _userManager.UpdateAsync(user);
        }
    }
    return RedirectToAction("List");
}
```

Фиг. 27 Метода "UpdateStatus (POST)"

3.5.3.8 Reservations

Извлича всички резервации на текущия вписан потребител, включително информация за конете и техните породи.

```
public IActionResult Reservations()
{
    List<Reservation> userReservation = _context.reservations
        .Include(x => x.Horse)
        .ThenInclude(x => x.Breed)
        .Where(x => x.User.UserName == User.Identity.Name)
        .ToList();
    return View(userReservation);
}
```

Фиг. 28 Метода "Reservations"

3.5.4 ReservationController

ReservationController управлява заявките, свързани с резервирането на коне. Той обработва създаването на резервации, проверява за наличност и осигурява защита срещу припокриващи се резервации.

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using HorseBase.Models;
using System.Linq;
using System.Threading.Tasks;
using HorseBase.Data;
using HorseBase.Models.ViewModels;

namespace HorseBase.Controllers
{
    public class ReservationController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ReservationController(ApplicationDbContext context)
        {
            _context = context;
        }
    }
}
```

Фиг. 29 Част от класа “ReservationController”

3.5.4.1 Create (GET)

Зарежда формата за резервация, като извлича информация за коня и създава модел за изгледа.

```
public async Task<IActionResult> Create(int horseId)
{
    var horse = await _context.horses.FindAsync(horseId);
    if (horse == null)
    {
        return NotFound();
    }

    var reservation = new ReservationViewModel
    {
        HorseId = horse.Id,
        Horse = horse,
        Price = 0
    };

    return View(reservation);
}
```

Фиг. 30 Метода “Create(GET)”

3.5.4.2 Create (POST)

Обработка подадените данни, проверява за съществуващи резервации в същия интервал и ако няма конфликт, записва нова резервация в базата.

```
[HttpPost]
public async Task<IActionResult> Create(ReservationViewModel reservationRequest)
{
    if (ModelState.IsValid)
    {
        // Fetch the horse details
        var horse = await _context.horses.FindAsync(reservationRequest.HorseId);
        if (horse == null)
        {
            return NotFound();
        }

        // Check for overlapping reservations
        bool isOverlapping = await _context.reservations.AnyAsync(r =>
            r.Horse.Id == reservationRequest.HorseId &&
            r.TakeHour < reservationRequest.ReturnHour &&
            r.ReturnHour > reservationRequest.TakeHour);

        if (isOverlapping)
        {
            ModelState.AddModelError("", "This horse is already booked for the selected time. Please choose another date.");
            return View(reservationRequest);
        }

        // Fetch the current user
        var user = await _context.Users.FirstOrDefaultAsync(x => x.UserName == User.Identity.Name);
        if (user == null)
        {
            return Unauthorized();
        }

        // Create the reservation
        Reservation reservation = new Reservation()
        {
            Horse = horse,
            Price = (double)reservationRequest.Price,
            TakeHour = reservationRequest.TakeHour,
            ReturnHour = reservationRequest.ReturnHour,
            UserId = user.Id
        };

        // Save the reservation to the database
        _context.reservations.Add(reservation);
        await _context.SaveChangesAsync();

        return RedirectToAction("Index", "Home");
    }

    return View(reservationRequest);
}
```

Фиг. 31 Метода “Create(POST)”

3.5.4.3 CheckOverlaps

Проверява дали избраният кон вече е резервиран за даден период и връща JSON отговор с резултата.

```
[HttpGet]
public async Task<IActionResult> CheckOverlaps(int horseId, DateTime takeHour, DateTime returnHour)
{
    bool isOverlapping = await _context.reservations.AnyAsync(r =>
        r.Horse.Id == horseId &&
        r.TakeHour < returnHour &&
        r.ReturnHour > takeHour);
    return Json(new { isOverlapping });
}
```

Фиг. 32 Метода “CheckOverlaps”

Част 4. Заключение

Проектът „Конна база“ представлява уебсайт, който предоставя на потребителите възможност за лесно разглеждане, управление и резервиране на коне за определени периоди от време. Основната цел на платформата е да създаде удобен, функционален и ефективен инструмент за потребителите, като същевременно улесни собствениците на конната база в управлението на наличните коне и резервации.

Разработената система използва архитектурата модел-изглед-контролер (MVC), която осигурява ясно разделение на отговорностите в кода, правейки го структуриран и лесен за поддръжка. Данните за конете, потребителите и резервациите се съхраняват в база данни, като се използва Entity Framework Core за взаимодействие с нея. Това позволява бързо и ефективно управление на информацията, както и защита на чувствителни потребителски данни.

Разработката на проекта е базирана на C# и ASP.NET Core, което осигурява висока производителност, сигурност и мащабируемост на системата. Използването на Razor View Engine за изгледите гарантира динамично зареждане на информацията и подобро потребителско изживяване. В допълнение, платформата е проектирана да бъде лесна за разширяване, което позволява добавяне на нови функционалности без сериозни промени в основната ѝ структура.

Създаденият уебсайт „Конна база“ е модерно и надеждно решение за управлението на резервации, осигуряващо удобен достъп до информация и подобро взаимодействие между потребителите и собствениците на конната база. Благодарение на добре структурираната архитектура, интуитивния потребителски интерфейс и стабилната технология, проектът предоставя ефективно и иновативно решение за управление на резервации, което може лесно да се адаптира към бъдещи нужди и подобрения.

Част 5. Използвани Източници

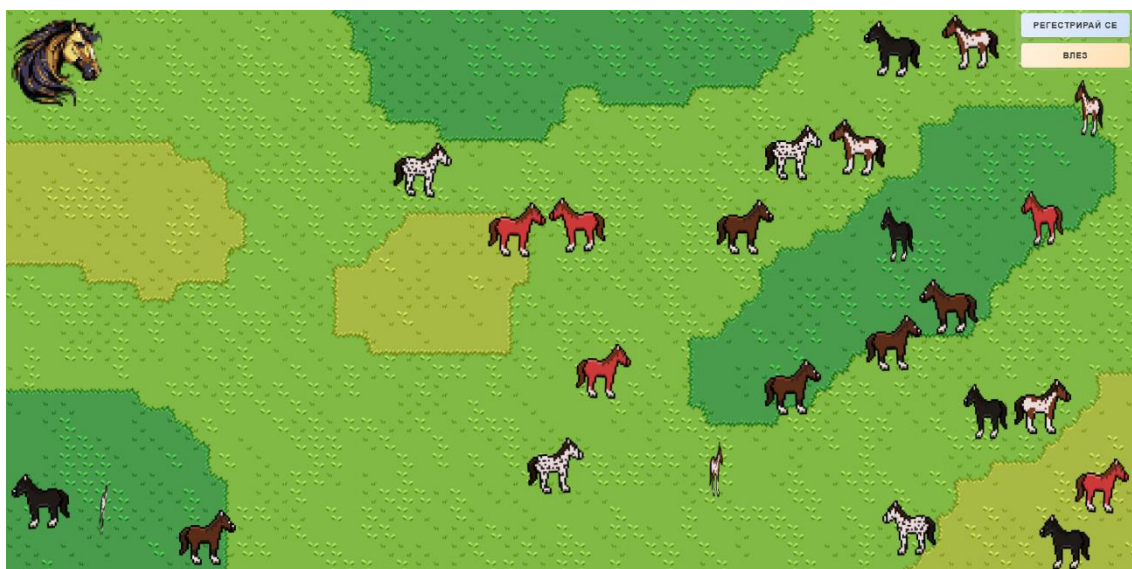
- <https://getbootstrap.com/>
- <https://dotnet.microsoft.com/en-us/apps/aspnet>
- <https://learn.microsoft.com/en-us/dotnet/csharp/>
- <https://www.w3schools.com/>
- [C# Guide - .NET managed language](#)
- [ASP.NET Core | Open-source web framework for .NET](#)

Част 6. Работа с приложението „Конна база“

6.1 Начална страница

Началната страница на уебсайта „Конна база“ представя интерактивен и визуално ангажиран дизайн, съчетаващ пиксел арт естетика с тематична визия, отразяваща основната идея на платформата. Централният фон изобразява природен пейзаж, включващ различни нюанси на зелено, жълто и кафяво, пресъздаващи ефекта на открито пасище. Върху тази среда са разположени множество коне в различни цветови вариации, които символизират богатото разнообразие от налични за разглеждане и резервиране животни. Всеки нарисован кон репрезентира съществуващ кон в базата данни.

В горната дясна част на страницата са позиционирани два основни бутона за навигация – „Регистрирай се“ и „Влез“. Те осигуряват на потребителите възможност да създадат нов акаунт или да влязат в съществуващ профил, което е ключов момент за достъпа до разширените функционалности на платформата.



Фиг. 33 Изглед на началната страница

6.2 Страница за регистрация

Страницата за регистрация позволява на потребителите да създадат нов акаунт, като въведат своите лични данни: потребителско име, имейл адрес, парола, собствено, бащино и фамилно име. За да се осигури висока сигурност, паролата трябва да отговаря на определени изисквания – да съдържа поне една цифра, една малка буква, една главна буква и специален символ. Ако въведената парола не покрива тези критерии, системата ще изиска промяна. Ако потребителят вече има създаден акаунт, той може да натисне хиперлинка „Влез“, който ще го отпрати в страницата за вход. След успешна регистрация, потребителите ще могат да влязат в своя акаунт и да използват предлаганите функционалности на сайта.



Регистриране

Потребителско Име
HorseMaister

Имейл
kon@kon.com

Парола
.....

Собствено име
Иван

Бащино име
Петров

Фамилно име
Георгиев

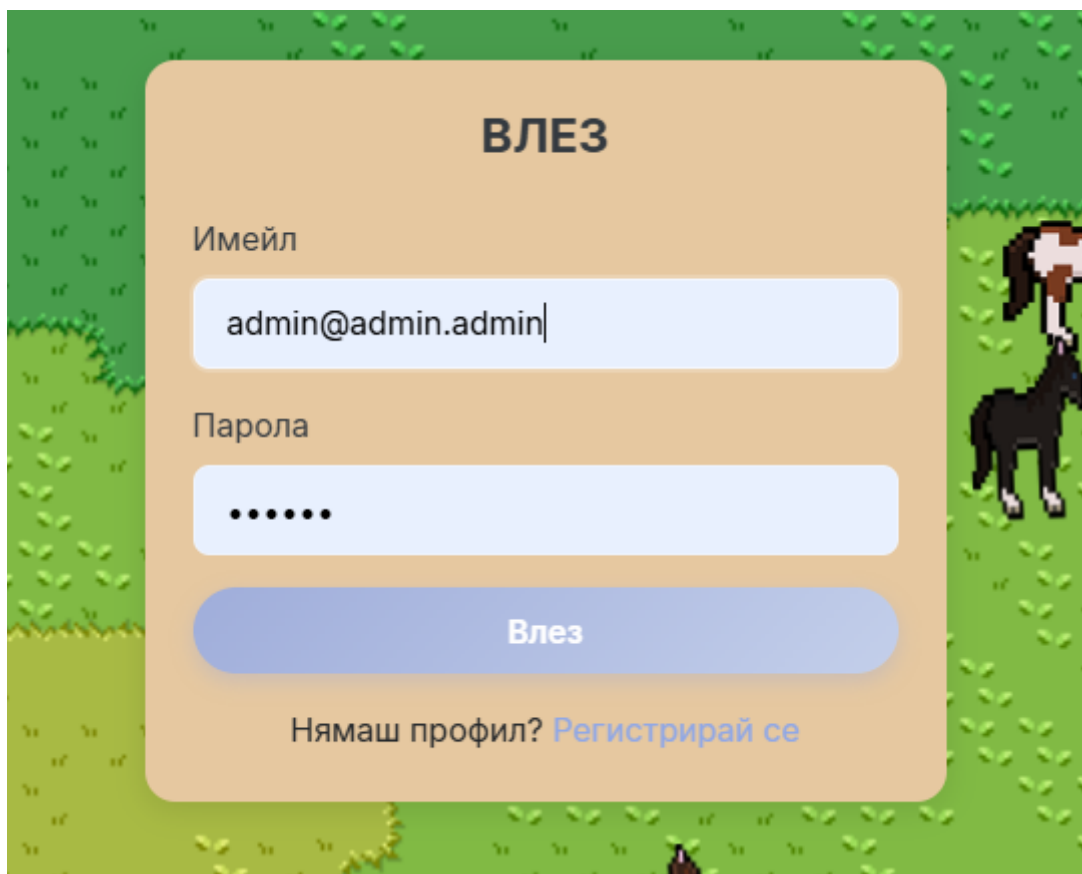
Регистрирай се

Вече имаш профил? [Влез](#)

Фиг. 34 Изглед на регистрационната форма

6.3 Страница за влизане

Страницата за влизане позволява на потребителите с вече създаден акаунт да получат достъп до системата. За да влязат, те трябва да въведат своя имейл адрес и парола. Ако потребителят все още няма акаунт, може да натисне хиперлинка „Регистрирай се“, който ще го отведе към страницата за регистрация.



The image shows a login form titled "ВЛЕЗ" (Login) centered on a light orange background. The form is set against a green field with a pixelated horse on the right. It contains two input fields: "Имейл" (Email) with the text "admin@admin.admin|" and "Парола" (Password) with masked characters ".....". Below the fields is a blue "Влез" (Login) button. At the bottom, it says "Нямаш профил? [Регистрирай се](#)" (Don't have a profile? [Register](#)).

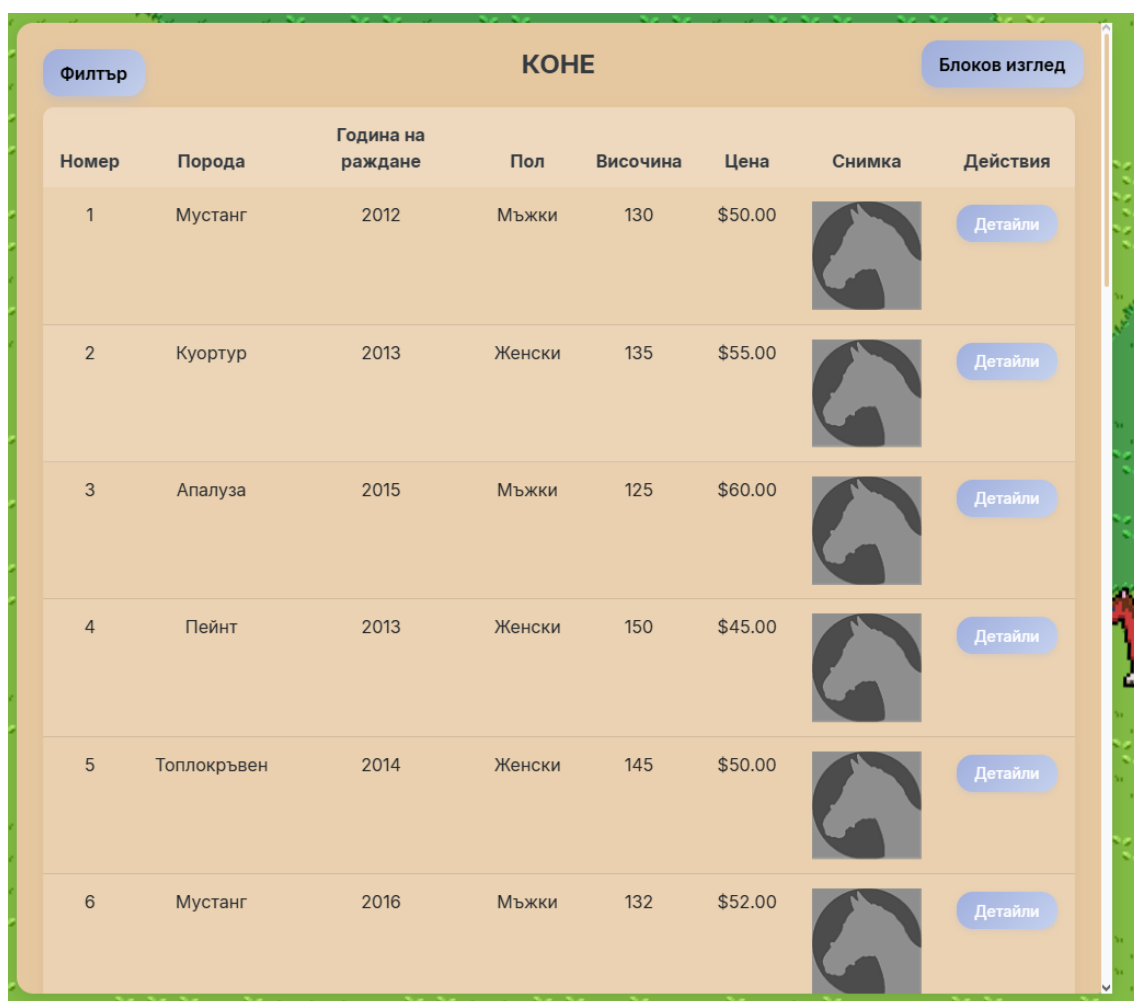
Фиг. 35 Изглед на входната форма

6.4 Лист с коне

При натискане на бутона „Разгледайте конете!“ на началната страница, потребителят ще бъде пренасочен към страница, в която може да разгледа всички коне, добавени в базата данни.

Конете са представени в табличен вид, като характеристиките им са организирани в колони, а всеки ред съдържа информация за отделен кон. Таблицата включва следните атрибути: номер, порода, година на раждане, пол, височина, цена за час и снимка. Снимката представлява първото изображение, качено от администратора, а ако такова не е налично, се използва шаблонно изображение.

Потребителят може да разглежда всички коне, като използва лентата за превъртане.



КОНЕ							
Филтър						Блоков изглед	
Номер	Порода	Година на раждане	Пол	Височина	Цена	Снимка	Действия
1	Мустанг	2012	Мъжки	130	\$50.00		Детайли
2	Куортур	2013	Женски	135	\$55.00		Детайли
3	Апалуза	2015	Мъжки	125	\$60.00		Детайли
4	Пейнт	2013	Женски	150	\$45.00		Детайли
5	Топлокръвен	2014	Женски	145	\$50.00		Детайли
6	Мустанг	2016	Мъжки	132	\$52.00		Детайли

Фиг. 36 Изглед на листата с коне (листов изглед)

Ако потребителят се интересува повече от външния вид на коня, отколкото от неговите характеристики, той може да избере да разглежда конете в блоков изглед, като натисне съответния бутон.

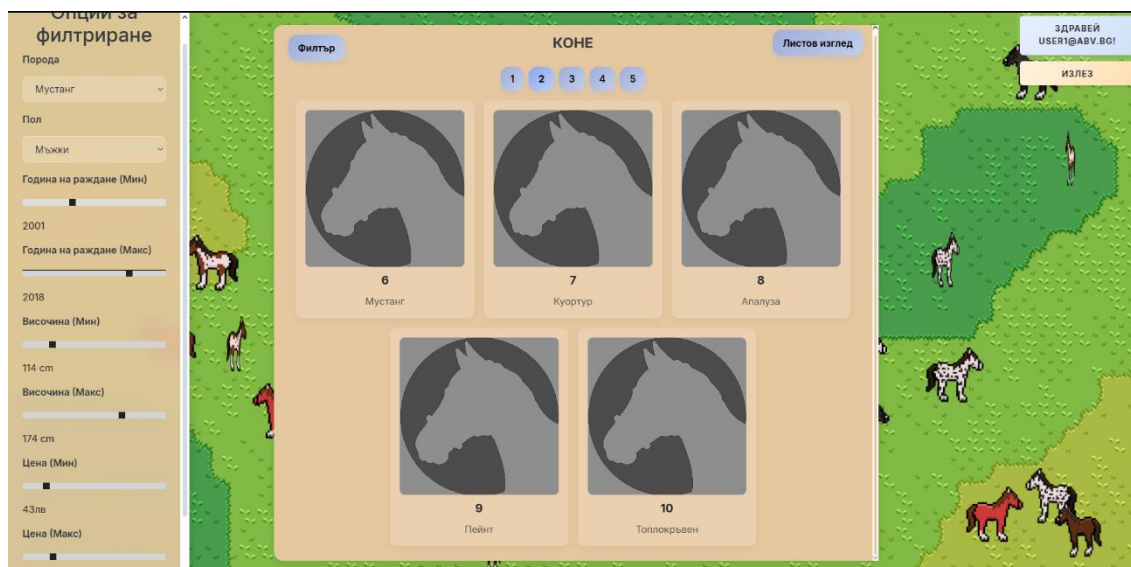
При активиране на този изглед конете ще бъдат представени в блокове, съдържащи само снимката, номера и породата на всеки кон. За да прегледа всички коне, потребителят трябва да използва странирането, като на една страница могат да бъдат показани максимум пет коня.



Фиг. 37 Изглед на листата с коне (блоков изглед)

За улеснение на потребителите, които търсят кон с определени характеристики, е внедрен филтър, който позволява търсене по порода, пол, възраст, височина и цена.

Филтърът е достъпен както в списъчния, така и в блоковия изглед. Той се отваря при натискане на бутона „Филтър“ и се затваря автоматично, когато се приложат избраните критерии или ако потребителят кликне с мишката извън него.



Фиг. 38 Изглед на листата с коне (блоков изглед) с филтър

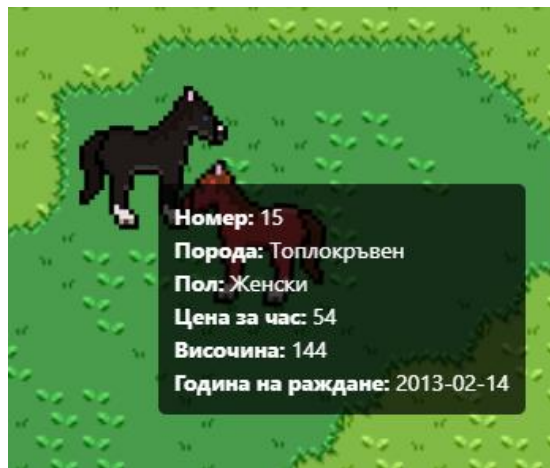
6.5 Детайли на коня

За да достигне страницата с детайлите на конкретен кон, потребителят има няколко възможности.

Ако използва списъчния изглед, той може да натисне бутона „Детайли“ към избрания кон. В блоковия изглед достъпът до детайлната страница става чрез кликуване върху самия блок на коня.

Допълнително, независимо от страницата, на която се намира, потребителят може да задържи курсора на мишката върху движещите се коне по екрана. В този случай, коня ще спре да се движи и в близост до позицията на курсора ще се появи малко информационно меню с детайли за съответния кон. Всеки кон притежава визуален изглед, съответстващ на породата му, което позволява на потребителя да добие представа за окраската му дори и при липса на снимка.

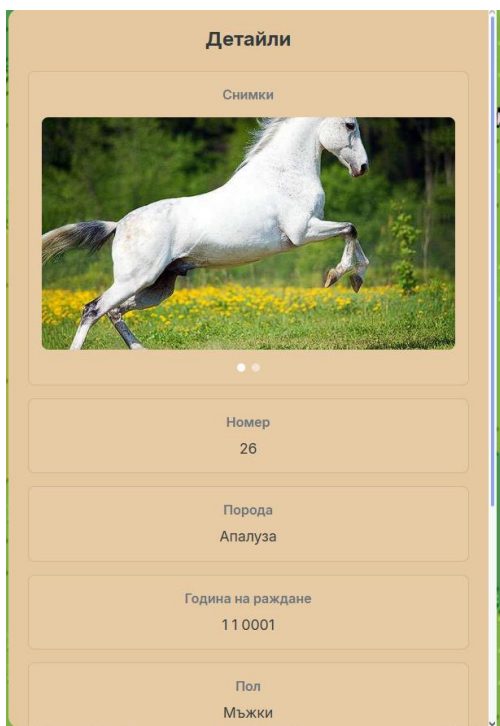
При кликуване върху избрания кон, потребителят ще бъде пренасочен към страницата с подробности за него.



Фиг. 39 Изглед на кон с информационно табло

На страницата с детайли потребителят ще има възможност да прегледа всички налични снимки на коня. Изображенията ще се сменят автоматично на всеки 7 секунди, като потребителят може също така да ги превключва ръчно.

Освен снимковия материал, цялата информация за коня ще бъде представена в страницата в структуриран и лесен за четене формат.



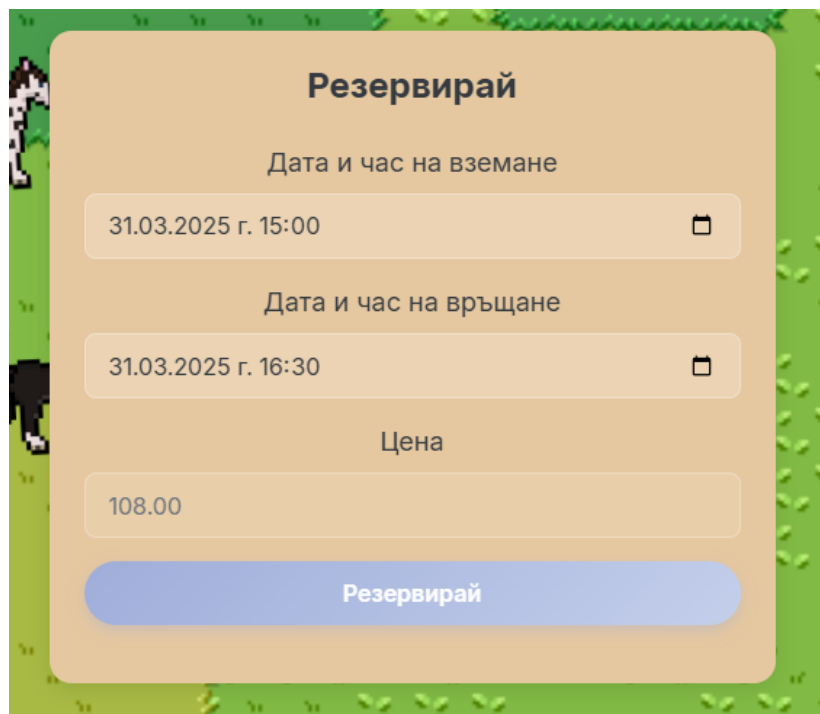
Фиг. 40 и 41 Изглед на детайли на кон

6.6 Резервация на кон

След като потребителят избере кон за резервиране, той трябва да натисне бутона „Резервирай“ в страницата с детайлите на коня. Това ще го пренасочи към страницата за резервация.

В тази страница потребителят трябва да въведе дата и час за вземане и връщане на коня. Системата автоматично ще изчисли крайната цена, която трябва да бъде заплатена.

Ако въведената информация е невалидна, например ако датата на вземане е по-късно от датата на връщане, ще се появи съобщение за грешка. В случай че конят вече е резервиран за избрания период, потребителят ще получи уведомление за невъзможността да направи резервацията.



The image shows a web form titled "Резервирай" (Reserve) for horse rental. The form is set against a green grass background with a small horse icon on the left. It contains three input fields: "Дата и час на вземане" (Pickup date and time) with the value "31.03.2025 г. 15:00", "Дата и час на връщане" (Return date and time) with the value "31.03.2025 г. 16:30", and "Цена" (Price) with the value "108.00". Each input field has a calendar icon on the right. At the bottom is a blue button labeled "Резервирай".

Фиг. 42 Изглед на регистрация на кон

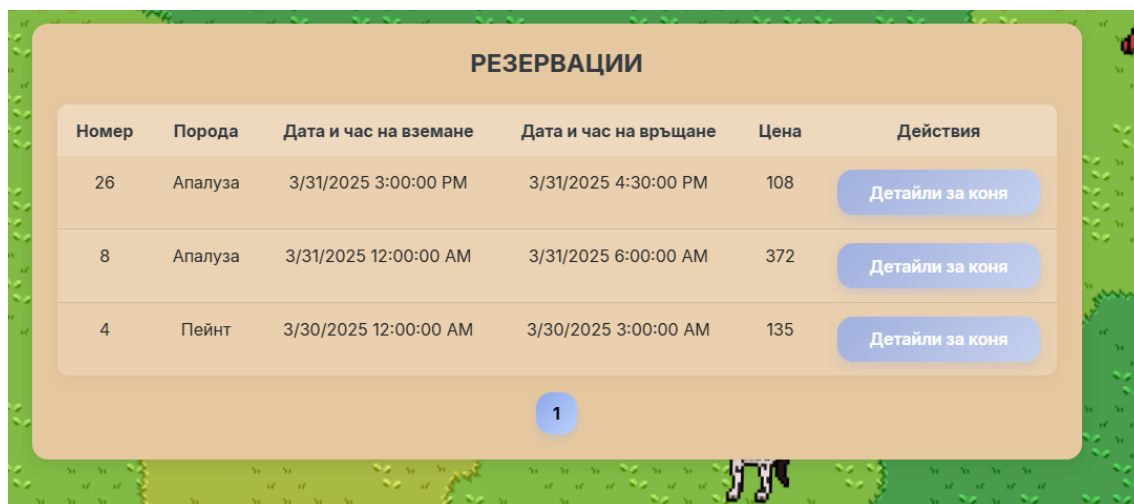
6.7 Управление на резервациите

След като потребителят направи резервация, тя автоматично се запазва в профила му.

Ако желае да прегледа своите резервации, той може да отвори профила си, където ще бъде достъпен списък с всички направени от него резервации. В този списък се показват времевият период на резервацията и съответната сума.

Освен това, потребителят има възможност да разгледа детайлите за коня, с който е направил резервацията, като натисне бутона „Детайли за коня“.

Резервациите са организирани в страници, като всяка страница може да съдържа до три резервации.



Номер	Порода	Дата и час на вземане	Дата и час на връщане	Цена	Действия
26	Апалуза	3/31/2025 3:00:00 PM	3/31/2025 4:30:00 PM	108	Детайли за коня
8	Апалуза	3/31/2025 12:00:00 AM	3/31/2025 6:00:00 AM	372	Детайли за коня
4	Пейнт	3/30/2025 12:00:00 AM	3/30/2025 3:00:00 AM	135	Детайли за коня

1

Фиг. 43 Изглед на управлението на резервации